CMPT 276: Assignment 4

Code Review

Brian Chan
Group 1
Simon Fraser University

March 31, 2026

1. Code Smells Identified in Teammate's Code

The following smells were identified during the review of Daniil's components:

- **Code-and-Paste Programming (`BankHeist.java`):** Hardcoded window dimensions (1600 and 896) were duplicated three times. This creates maintenance issues if the resolution needs to be updated in the future.
- **Primitive Obsession (`BankHeist.java`):** The use of the magic number `3` in `setDefaultCloseOperation()` instead of the standard `JFrame.EXIT_ON_CLOSE` constant.
- **Long Parameter List (`Cell.java`):** The constructor required four separate primitives (`x, y, width, height`), which is better handled by passing `Position` and `Dimension` objects.
- **Anti-Pattern (`Thief.java`):** The class implemented custom logic to check if the player was inside the board, duplicating a service already provided by the `Board.isInside()` method.

---

2. Steps Taken to Address Code Smells and Refactoring Performed

1. Duplicated Code

Location: `Interactable.java` and subclasses

- **The Smell:** Subclasses repeated the exact same code at the end of their collection logic (`this.isCollected = true;`). This lacked an idempotency guard, meaning simultaneous collisions could trigger effects twice.
- **The Refactoring:** I moved `onCollect` to the base class as a `public final` method that checks for prior collection before calling a new `applyEffect(board)` method.

2. Alternative Classes with Different Interfaces

Location: `CashStack.java`, `InkDart.java`, `RareDiamond.java`

- **The Smell:** Value-carrying subclasses used inconsistent naming for the same concept (e.g., `penalty` vs. `value`), increasing cognitive load.

- **The Refactoring:** I standardized variables to `private final int value` across all related classes to ensure consistent internal representation and proper encapsulation.

3. Primitive Obsession

Location: Multiple `Interactable` subclasses

- **The Smell:** Item effects were hard-coded as raw integers (e.g., 100, -150), acting as "magic numbers" that are difficult to adjust or understand at a glance.
- **The Refactoring:** These values were extracted into named constants like `FREEZE_DURATION_TICKS` and `REWARD_VALUE` for better self-documentation.

4. High Coupling

Location: `RareDiamond.java`, `Board.java`

- **The Smell:** `RareDiamond` was directly calling `board.restartDiamondTimer()`, creating "Inappropriate Intimacy" where an item class controlled the board's internal timing logic.
- **The Refactoring:** The timer reset logic was moved back to the `Board` class, ensuring the diamond only handles its reward while the board manages the game state.

Testing Verification

All refactorings were verified using the existing test suite via `mvn clean test`. For internal changes like renaming variables and extracting constants, all tests passed immediately. The fix for Duplicated Code required updating `testMultipleCollections` to use distinct object instances to properly simulate multiple collisions rather than asserting non-idempotent behavior. For High Coupling and the redundant Anti-Pattern logic, integration tests confirmed that shifting logic to the appropriate classes maintained correct game behavior. All tests now pass, confirming a more modular and stable system.